

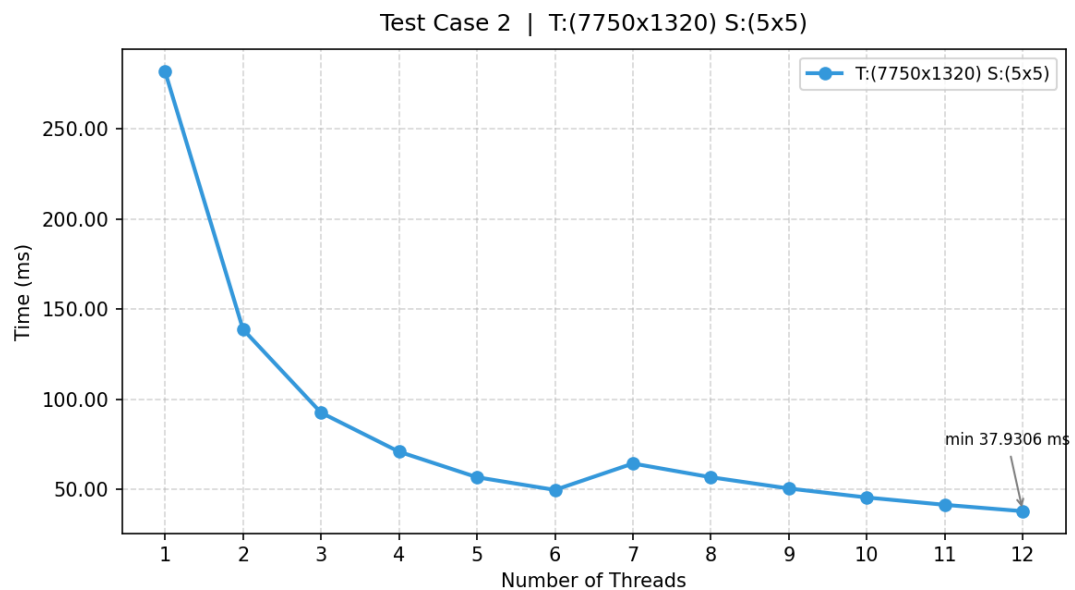
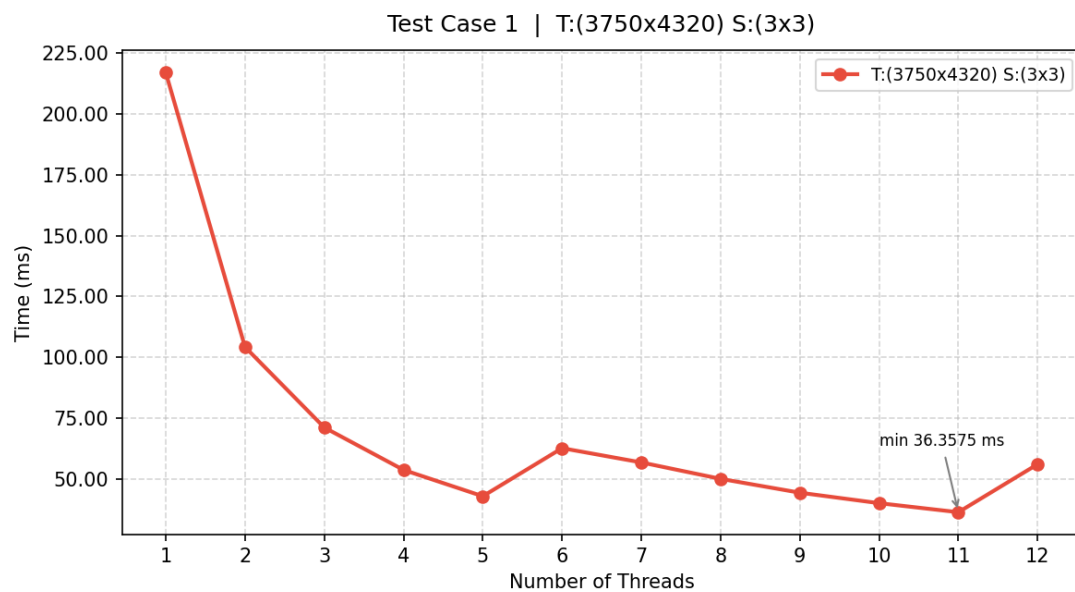
Template Matching

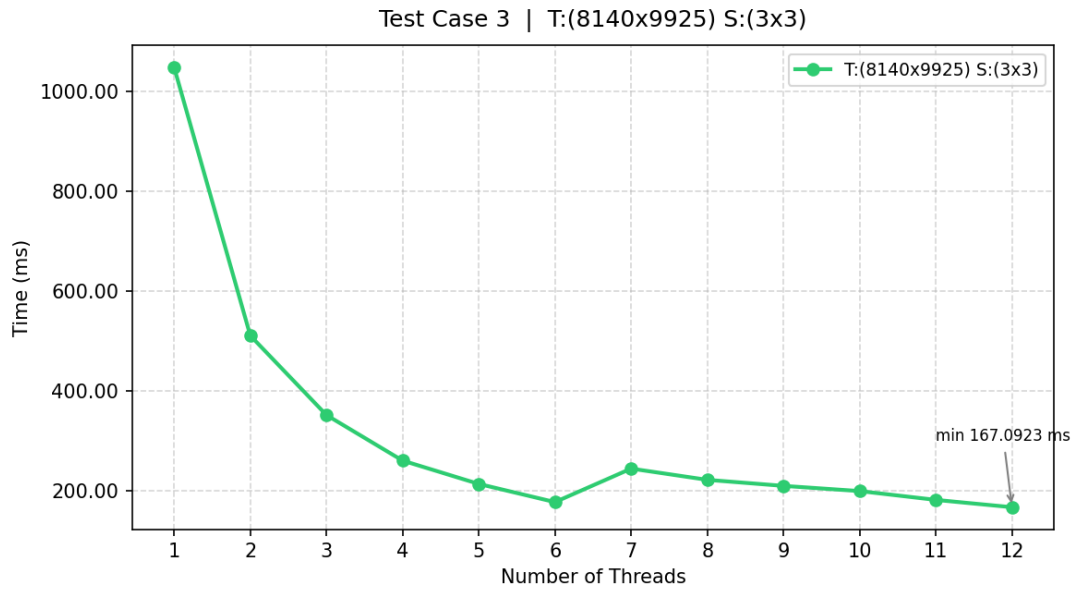
A1125515-邢立承

g++ -O2 -march=native -fopenmp -pthread -o test_all_cpu test_all_cpu.cpp -lm

不同執行緒數量（1 至 12 個 Thread）下，完成

Template Matching 所需時間的折線圖





除了 Block Size 的參數之外，你還有做什麼特別的效能優化嗎？

0. 生產者—消費者模式（不影響計時解果）

原本的程式是「讀取測資 1 → 運算測資 1 → 讀取測資 2 → ...」。

現在優化為：**背景執行緒（生產者）**負責把耗時的檔案 I/O（讀取巨大的 T 矩陣 text 檔）預先讀到記憶體中放入 Queue；而 **主執行緒（消費者）**則專心執行 CPU 密集型的 OpenMP 計算。不會影響單筆測資的 NCC 計算時間，只會影響

全部測資跑完的總時間

1. OpenMP 平行化 (collapse(2))

用 `#pragma omp parallel for num_threads(num_threads) collapse(2)`

將 (r,c) 的二維搜尋空間展平成一個大迴圈，提高工作分配均勻度

2. Template 轉成 float 快取

原本的 S 是 `unsigned char`，在與浮點數做乘法和加法時，編譯器會在內層迴圈不斷進行整數到浮點數的轉型（Type promotion）。

在迴圈外宣告了一個浮點數版本的 S 陣列 `float* Sf = (float*)malloc(n*sizeof(float));`，這樣在最內層的雙層迴圈中，讀取 x 時就直接是 `float`，節省了巨量的 CPU 週期

作業遇到的錯誤

使用 `#pragma omp simd reduction(+:sumX...)` 反而變慢

測資中，Template S 的大小通常只有 3x3 或 5x5。

`#pragma omp simd` 會強程式用向量指令打包，但打包向量暫存器（Setup）和最後加總合併結果（Horizontal Reduction）是需要額外時間成本的。當迴圈只有 3 或 5 次時，執行 SIMD 指令的「前置準備手續費」遠遠大於直接用普通運算算完這 3 個數字的時間，結果平均多使用了 0.002ms

同樣 SIMD 也是遇到相同問題無法優化，不過在命令列加入 `-march=native` 確實能讓 OpenMP 更快

這次作業的難度?或是其他建議

這次作業複雜度太簡單了，無法使用投影片當中所教到的技巧，例如 T 和 S 在 `load_matrix()` 之後就再也不會被修改，`matchCPU()` 裡面沒有任何寫入操作，OpenMP 的執行緒本來就可以同時讀取，完全不需要 Reader-Writer Lock，也因為沒有需要 mutex 爭搶，如果強行加入 `pthread_cond_wait` 反而會更慢，還有測試資料太小不會造成溢位，不需要手動調整堆疊大小等等

心得感想

在這次的矩陣比對程式優化過程中，我對平行運算有了更深層的認識。最大的收穫不是學到哪個技術能加速，而是學會了**如何判斷一個方法為什麼沒用**。

起初，直覺認為這類密集的影像區域匹配只能仰賴 GPU 的龐大核心數來暴力破解。但在實作 CPU 版本的過程中，我實踐了許多底層優化技巧

一開始我直覺認為，只要在程式裡加入更多同步機制，例如 Reader-Writer Lock、條件變數、調整執行緒優先權，就能讓程式跑得更快。但實際分析後才發現，這些工具解決的是「執行緒之間如何協調」的問題，而我程式的瓶頸根本不在協調，而在於記憶體頻寬。T3 測資的矩陣將近 80MB，遠超過 L3 Cache 的容量，CPU 大部分時間都在等待資料從 RAM 搬進來，無論執行緒排程得多精巧，RAM 的物理速度就是上限，這是任何軟體層面的協調機制都無法突破的。

這讓我理解到一個重要觀念：**工具要用在它設計的問題上才有意義**。
rwlock 適合讀多寫少且資料會動態更新的場景；cond_wait 適合生產者與消費者速度不對等的串流情境；排程優先權適合多程式競爭 CPU 資源的環境。把這些工具套用在靜態、一次性載入的矩陣比對上，不只沒有幫助，反而增加了不必要的開銷。